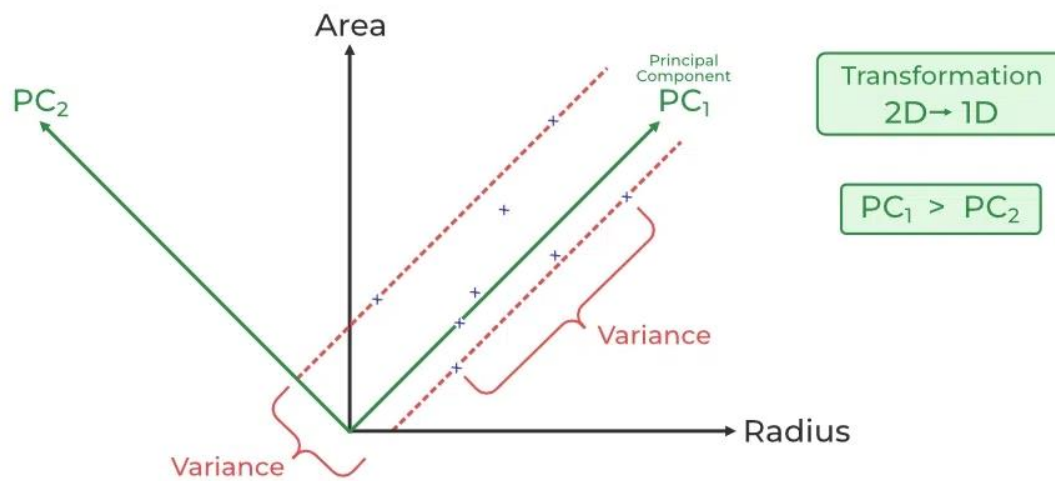


Principal Component Analysis (PCA)

Principal Components are **new features (axes/directions)** that PCA creates from your original dataset.

They:

- Capture the **most important patterns (variations)** in the data.
- Help reduce **dimensionality** — meaning you can describe your data with fewer variables while keeping most of the information.



PCA's goal

PCA's goal is to **summarize the data** using fewer features —but we want to **lose as little information as possible**.

So, we need to find the direction (line) where the data varies the most — because that's where the information is richest.

Original features: [Age, Salary, Experience, Spending Score]

After PCA: [PC1, PC2]

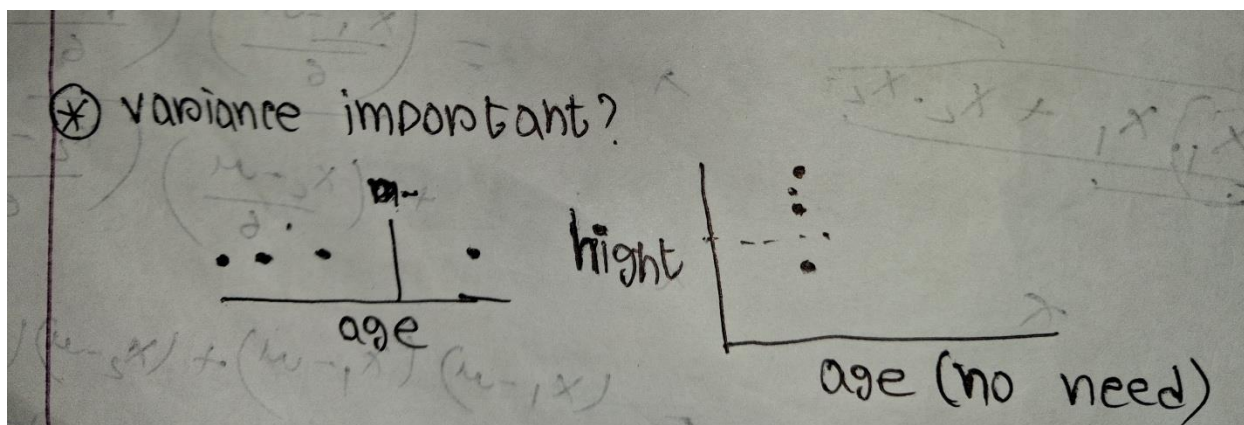
When to Use PCA in EDA

✓ Use PCA when:

- You have **many correlated features** (like 10+ features).
- You want to **visualize high-dimensional data** in 2D or 3D.
- You want to **detect structure, patterns, or clusters** in your data.
- You want to **reduce noise** and simplify models.

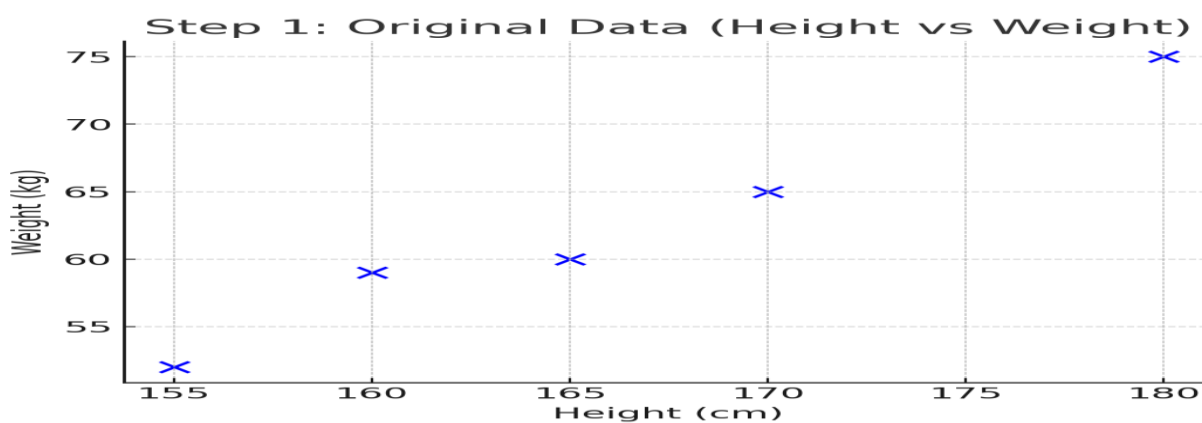
✗ Avoid PCA when:

- Features are already few and interpretable.
- You need feature interpretability (PCA transforms features, so meaning is lost).



We take a new basis based on variance (PCA) not to create a decision boundary, but to make the data representation better so that supervised models can learn decision boundaries more easily.

Problem:



Example Dataset –

Suppose we have **5 students** with two features: **Height (cm)** and **Weight (kg)**.

Student Height Weight

A	170	65
B	160	59
C	180	75
D	155	52
E	165	60

Step 1 — Step 1: Construct the Data Matrix

$$X = \begin{pmatrix} 170 & 65 \\ 160 & 59 \\ 180 & 75 \\ 155 & 52 \\ 165 & 60 \end{pmatrix}$$

n=5 samples, d=2 features

Vector : Vector is a list of number that has both magnitude(size) and direction.

Example $A = \begin{pmatrix} 170 \\ 65 \end{pmatrix}$

Step 2: Standardize / Center the Data (also called z-score normalization)



Equation:

$$z = \frac{x - \mu}{\sigma}$$

Where:

- x = original value
- μ = mean of the feature/column
- σ = standard deviation of the feature/column
- z = standardized (scaled) value

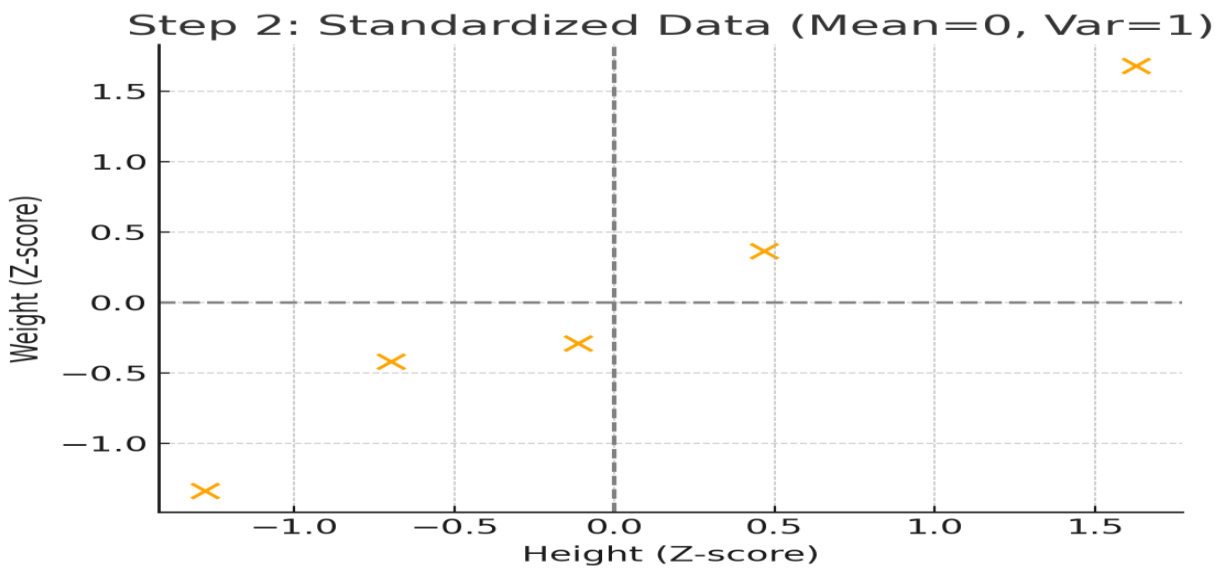


For each feature (column):

- Subtract the mean → so its average becomes 0.
- Divide by standard deviation → so its spread (variance) becomes 1.

Result:

→ All features are on the same scale (mean = 0, std = 1).



Why ?

- All your features are in the **same units and similar ranges** so we can **Unit-free comparison**.
- Variance is measured **fairly**, without being dominated by large-scale features.
- PCA now finds the **true pattern** in data, not just the biggest numbers.

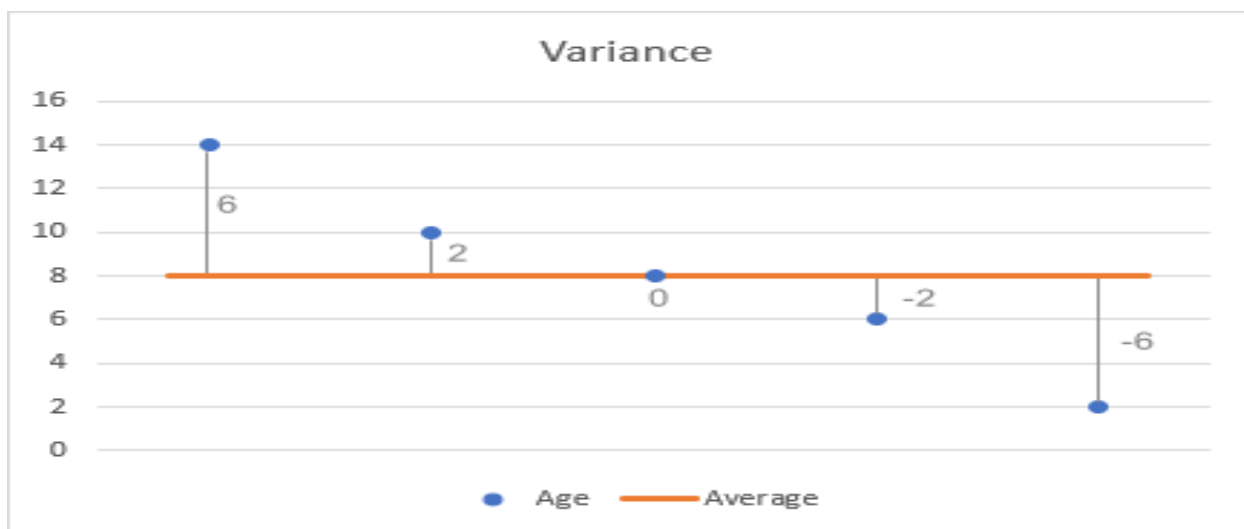
Step 3: Compute Covariance Matrix:

Variance: Show how much a single variable spreads out from its mean

Formula:

$$\text{Variance} = \frac{(X - \mu)^2}{N}$$

Visualization:

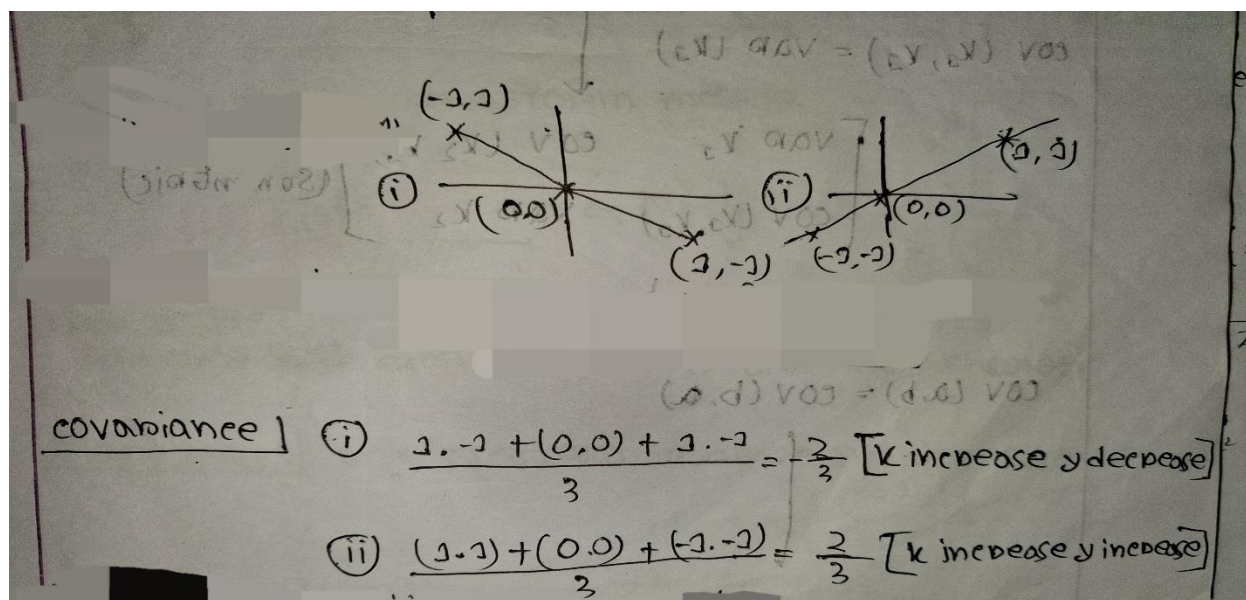


Note: Variance can never be negative

Covariance: Show How two variables change together.

Formula:

$$\text{Cov}(x,y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N}$$



Here the variance is the same, but the covariance is different because the direction is different. In this way, covariance shows the correlation between two variables

Covariance can be positive, negative, or zero.

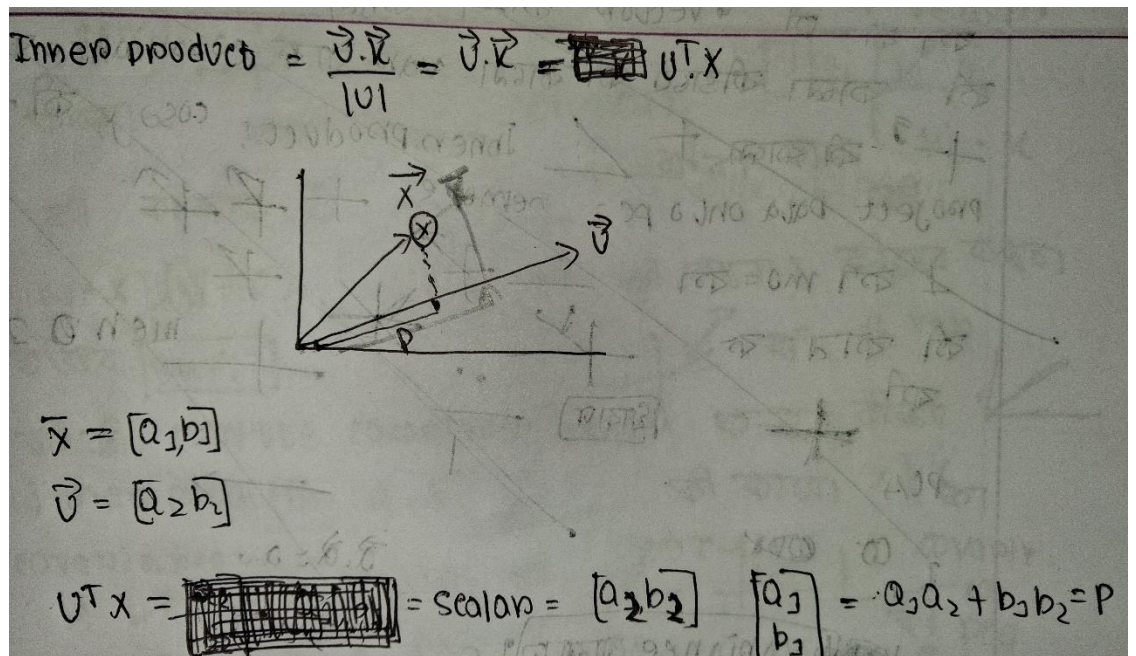
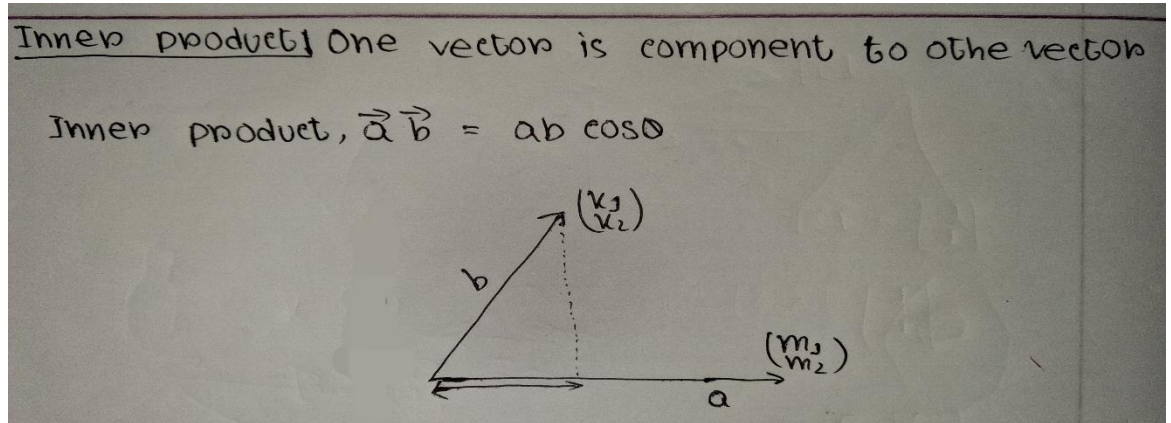
4. Sign of Covariance

Covariance sign	Meaning	Example
Positive	Both variables increase/decrease together	Height $\uparrow \rightarrow$ Weight $\uparrow$
Negative	One increases while the other decreases	Speed $\uparrow \rightarrow$ Travel time $\downarrow$
Zero	No consistent relationship	Height $\leftrightarrow$ Shoe color

1. Basic Meaning

Concept	What it measures	Type of measure
Variance	How much a <i>single variable</i> spreads out from its mean	Single-variable (self-spread)
Covariance	How <i>two variables</i> change together	Two-variable (relationship)

Inner Product:



In PCA, the inner product is used, which indicates how much of one vector lies in the direction of another vector. Here, the two vectors act as unit vectors

Input Features

Create Matrix

$$\begin{pmatrix} y_1 \\ p_1 \\ q_1 \end{pmatrix} \begin{pmatrix} y_2 \\ p_2 \\ q_2 \end{pmatrix} \begin{pmatrix} y_3 \\ p_3 \\ q_3 \end{pmatrix} = \begin{bmatrix} y_1 & y_2 & y_3 \\ p_1 & p_2 & p_3 \\ q_1 & q_2 & q_3 \end{bmatrix} = X$$

$$X^T X = \begin{bmatrix} y_1 & p_1 & q_1 \\ y_2 & p_2 & q_2 \\ y_3 & p_3 & q_3 \end{bmatrix} \begin{bmatrix} y_1 & y_2 & y_3 \\ p_1 & p_2 & p_3 \\ q_1 & q_2 & q_3 \end{bmatrix} = \text{Inner product.}$$

When a vector takes the inner product with itself, it indicates the variance, and when one line (vector) takes the inner product with another line, it means calculating the covariance

Covariance Matrix

$$\begin{bmatrix} \text{Var}(x_1) & \dots & \text{Cov}(x_n, x_1) \\ \vdots & \ddots & \vdots \\ \text{Cov}(x_n, x_1) & \dots & \text{Var}(x_n) \end{bmatrix}$$

What:

The **covariance matrix** measures how features vary *together*.

This matrix summarizes:

- Variance of each feature on the diagonal
- Covariance (relationship) on the off-diagonals

Formula used in PCA:

$$C = \frac{1}{n-1} Z^T Z$$
$$C = \begin{bmatrix} 1.0 & 0.986 \\ 0.986 & 1.0 \end{bmatrix}$$

Way of Covariance Matrix Calculation:

1 Manual calculation (statistical formula)

When you calculate **variance or covariance manually**, you usually use formulas like:

$$\text{Cov}(x, y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N}$$

These **already** subtract the mean inside the formula. So even if your data isn't mean-centered, the formula compensates — you'll get the correct covariance. So yes, manual formula is fine even without centering the data first.

2 Matrix shortcut (used in PCA)

When we use matrix form in PCA:

$$C = \frac{1}{n - 1} Z^T Z$$

This assumes: X is already mean-centered. Because it multiplies raw feature values directly, if you didn't center, those big mean values produce **extra cross-terms** like which artificially inflate covariance and variance.

Example

$$X = \begin{pmatrix} 2 \\ 4 \\ 6 \end{pmatrix} \quad Y = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

- **Matrix without centering:**

$$C = \begin{pmatrix} 18.67 & 9.33 \\ 9.33 & 4.67 \end{pmatrix}$$

⊗ Values inflated because mean not removed.

But after centering:

$$C = \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix}$$

Why PCA specifically cares

PCA doesn't compute each covariance manually.

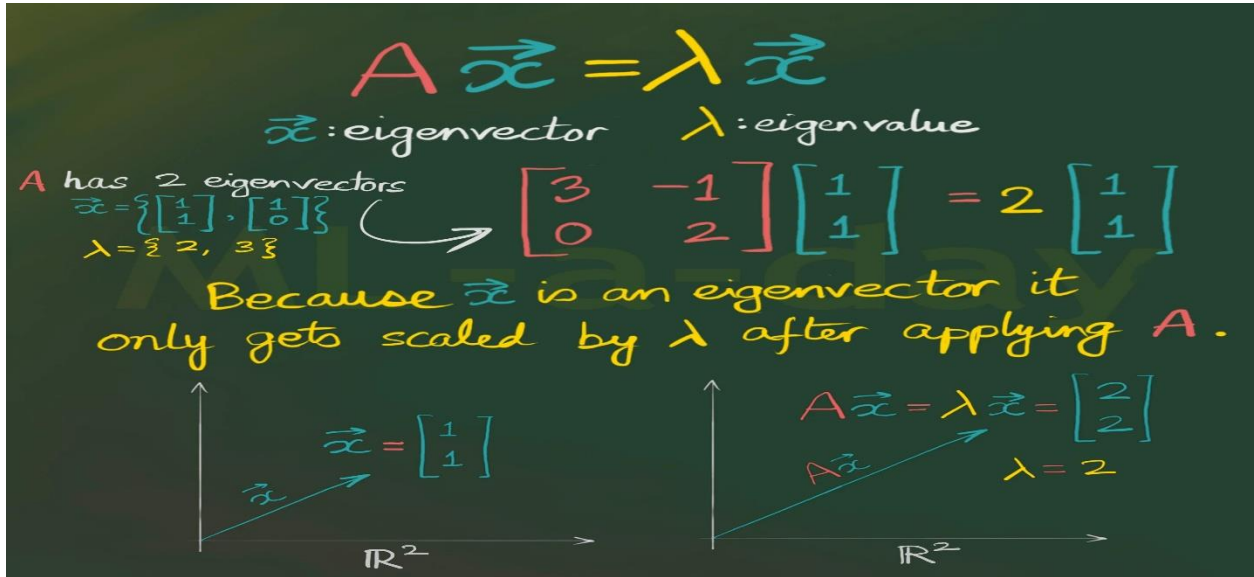
It uses **matrix multiplication** to get the covariance matrix fast.

So if your data isn't centered first:

- PCA thinks the mean is part of the "spread".
- Eigenvectors (principal directions) point toward the **mean offset**, not the **true direction of variation**.

Step 4 — Compute Eigenvalues and Eigenvectors

✓ What:



$A \vec{x} = \lambda \vec{x}$

$\vec{x}$: eigenvector λ : eigenvalue

A has 2 eigenvectors
 $\vec{x} = \left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right\}$
 $\lambda = \{ 2, 3 \}$

$\begin{bmatrix} 3 & -1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

Because $\vec{x}$ is an eigenvector it only gets scaled by λ after applying A .

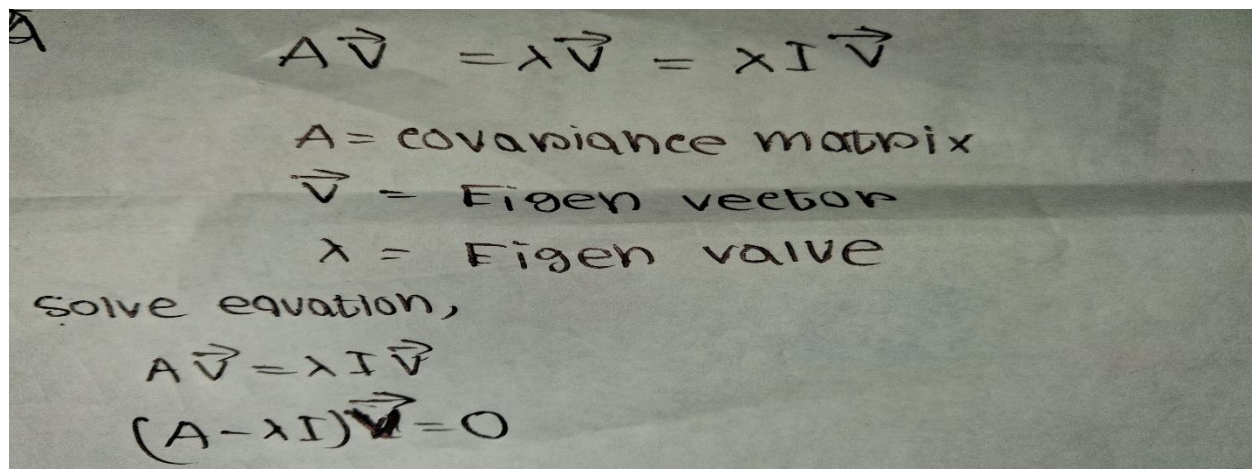
$\vec{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

$A \vec{x} = \lambda \vec{x} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$
 $\lambda = 2$

Imagine v is a vector (arrow) in space:

- Normally, multiplying a vector by a matrix **rotates and stretches** it.
- But eigenvectors are **special directions** that do **not rotate** under the transformation.
- The eigenvalue tells **how much the vector stretches** along that direction.

How?



Handwritten notes on a piece of paper:

$$A\vec{v} = \lambda\vec{v} = \lambda I\vec{v}$$

A = covariance matrix
 $\vec{v}$ = Eigen vector
 λ = Eigen value

Solve equation,

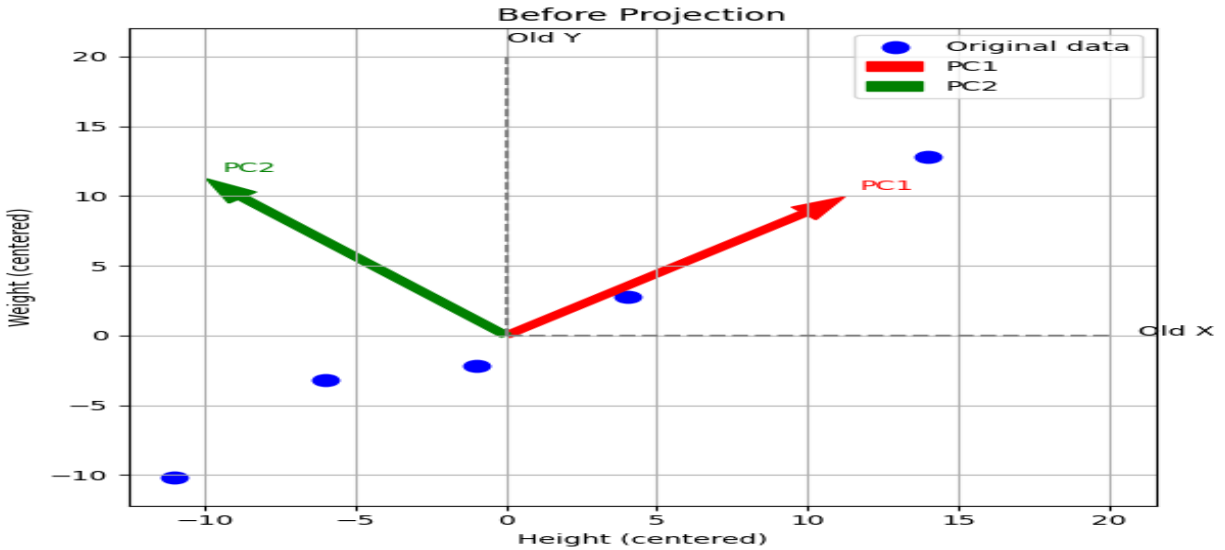
$$A\vec{v} = \lambda I\vec{v}$$
$$(A - \lambda I)\vec{v} = 0$$

Now,

$$\text{Det}(A - \lambda I) = 0$$

$$\lambda_1 = 1.986 \text{ and } \lambda_2 = 0.014$$

$$v_1 = [0.707, 0.707] \text{ and } v_2 = [-0.707, 0.707]$$



Why:

Each **eigenvector** represents a **new direction (Principal Component)**, and the **eigenvalue** tells how much variance that direction captures.

Sort eigenvalues (and their eigenvectors) in descending order.

$$\lambda_1 > \lambda_2$$

So: PC1 corresponds to v_1

PC2 corresponds to v_2

- $v_1 \rightarrow$ direction of maximum spread (most information)
- $v_2 \rightarrow$ direction of minimum spread

Because We want to keep the directions with **most variance** — they represent the data best

📌 Step 6 — Compute Explained Variance Ratio

What:

$$EVR_1 = \frac{1.986}{1.986 + 0.014} \approx 0.993$$
$$EVR_2 = 0.007$$

Why:

This tells **how much of the total information** is captured by each component.

Here PC1 alone explains **99.3%** of data variation → can reduce to **1 dimension** safely

Step 6: Project Data onto PC

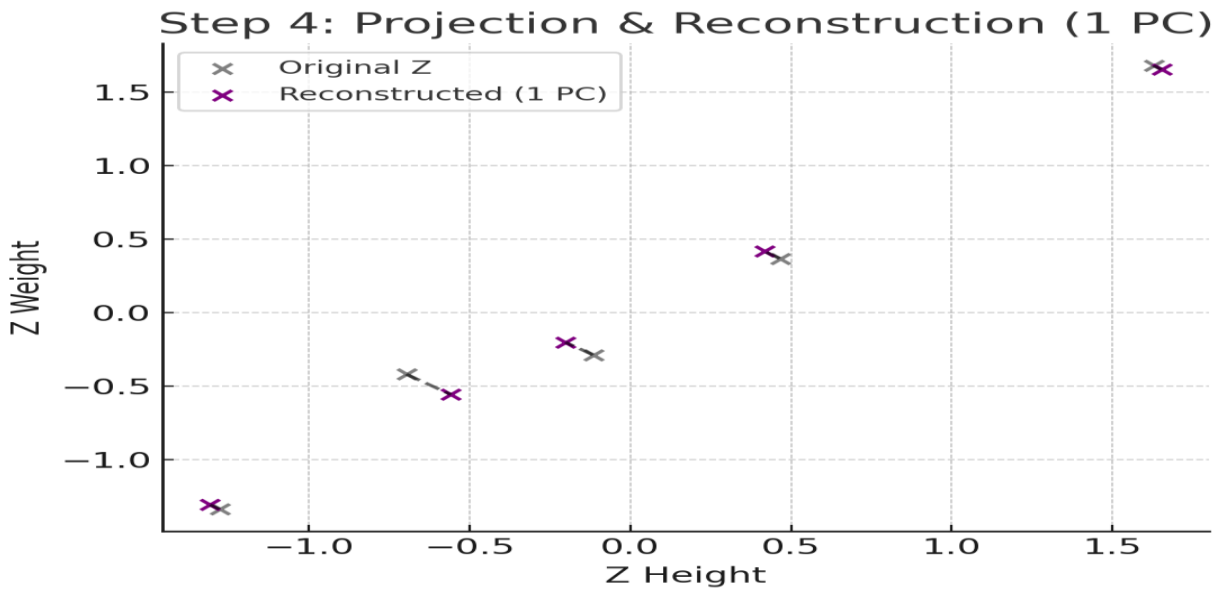
Formula: $X_{PCA} = Z \cdot V_1$

$$Z = X \begin{bmatrix} v_1 & v_2 \end{bmatrix} = \begin{bmatrix} \text{PC1} & \text{PC2} \end{bmatrix}$$

$$\begin{bmatrix} 0.522 & -0.062 \\ -0.708 & 0.175 \\ 2.123 & 0.033 \\ -1.672 & -0.037 \\ -0.254 & -0.108 \end{bmatrix}$$

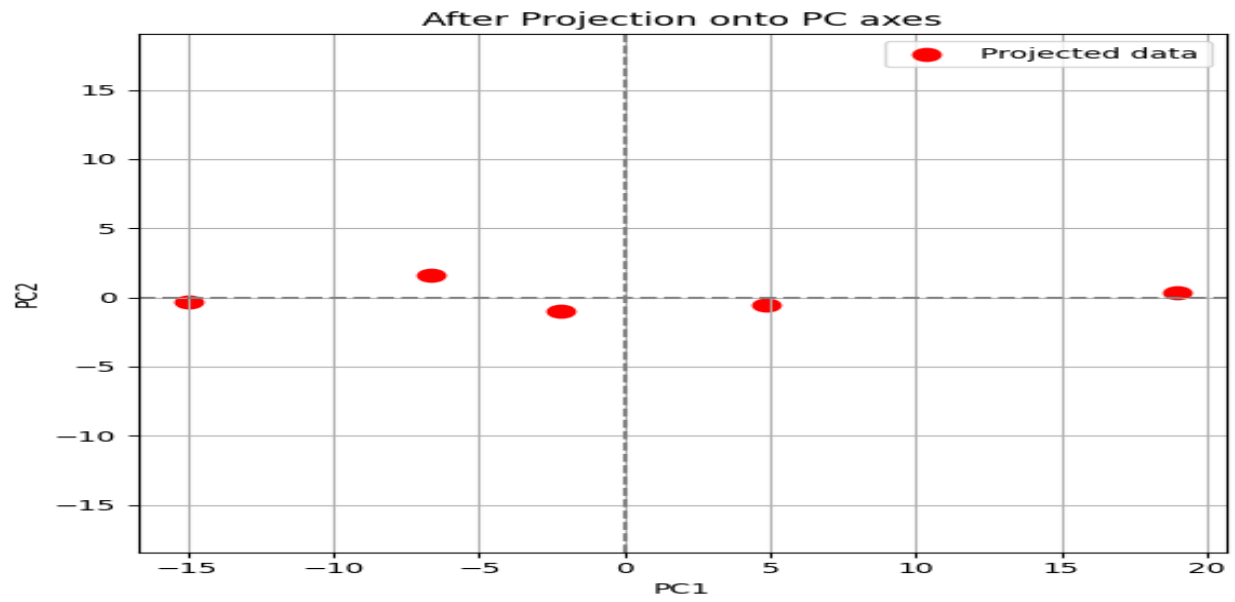
$$= \begin{bmatrix} 0.416 & 0.328 \\ -0.624 & -0.376 \\ 1.456 & 1.502 \\ -1.144 & -1.197 \\ -0.104 & -0.258 \end{bmatrix} \cdot \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix} = \begin{bmatrix} 0.522 \\ -0.708 \\ 2.123 \\ -1.672 \\ -0.254 \end{bmatrix}$$

here Z =Data matrix, V_1 =Eigen vector



Why:

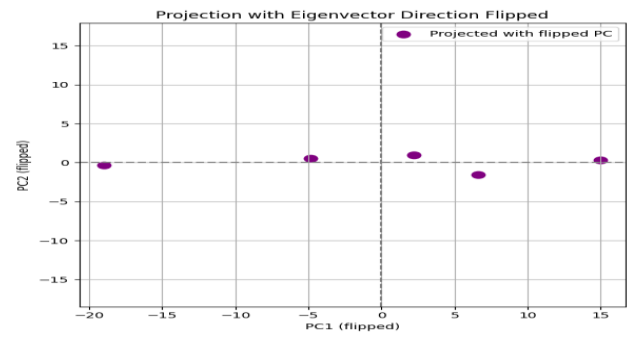
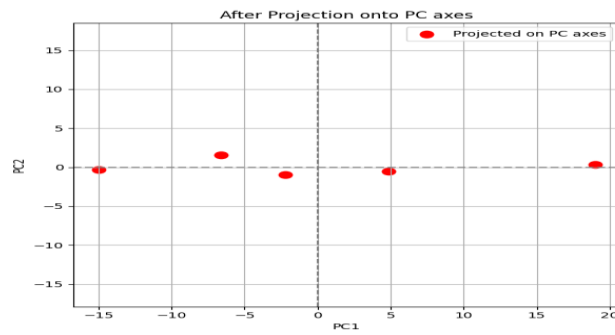
We project data onto eigenvectors because **eigenvectors define the directions of maximum information**, and projection converts the data into new uncorrelated features (principal components) that allow dimensionality reduction, noise removal, and reconstruction.



So projection removes correlation **because PCA rotates the coordinate system into a basis where features become statistically independent** (uncorrelated). So we can easily remove feature.

$$\text{Cov}(Z) = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

If Eigen vector directions change?



Eigenvector (v) and if ($-v$)

1) PCA axes remain the same

2) variance is the same

3) covariance is zero